

## **Daily Electricity Load Forecasting - Great Britain**

URN: 6926172

AI and Sustainability: EEEM075

MSc People-Centred Artificial Intelligence (Online)

University of Surrey

31<sup>st</sup> July 2026

## Table of Contents

|          |                                                                                 |           |
|----------|---------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b><i>Introduction</i></b>                                                      | <b>4</b>  |
| 1.1      | Sustainability Challenge and Motivation                                         | 4         |
| 1.2      | Problem Context                                                                 | 4         |
| 1.3      | Aims and Objectives                                                             | 4         |
| 1.4      | Data Review                                                                     | 5         |
| 1.5      | Report Structure                                                                | 5         |
| <b>2</b> | <b><i>Literature Review</i></b>                                                 | <b>5</b>  |
| 2.1      | Factors Influencing Electricity Demand                                          | 5         |
| 2.2      | UK Electricity Demand Under Disruption: The COVID-19 Case                       | 5         |
| 2.3      | Forecasting Model Approaches for Electricity Demand                             | 6         |
| 2.4      | Explainability in Load Forecasting                                              | 6         |
| 2.5      | Sustainable AI: Model Compression and Efficiency                                | 6         |
| 2.6      | Summary and Positioning of This Study                                           | 7         |
| <b>3</b> | <b><i>Data Understanding, Pre-processing, and Exploratory Data Analysis</i></b> | <b>7</b>  |
| 3.1      | Data Sources and Data Dictionary                                                | 7         |
| 3.1.1    | Data sources:                                                                   | 7         |
| 3.1.2    | Data Dictionary (merged dataset, before feature engineering):                   | 8         |
| 3.2      | Data Preparation and Merging                                                    | 8         |
| 3.3      | Exploratory Analysis of Demand Drivers                                          | 9         |
| 3.3.1    | Understanding the Variables                                                     | 9         |
| 3.3.2    | Calendar Effects                                                                | 10        |
| 3.3.3    | Weather Relationships                                                           | 11        |
| 3.3.4    | Economic Indicators                                                             | 12        |
| 3.3.5    | COVID-19 Disruption                                                             | 13        |
| 3.4      | Cross-Predictor Checks                                                          | 14        |
| 3.5      | Feature Engineering                                                             | 14        |
| 3.6      | Train/Validation/Test Split                                                     | 15        |
| 3.7      | Scaling and Normalisation                                                       | 16        |
| <b>4</b> | <b><i>Modelling</i></b>                                                         | <b>16</b> |
| 4.1      | Problem Formulation                                                             | 16        |
| 4.2      | Model Selection Rationale                                                       | 16        |
| 4.3      | XGBoost: Hyperparameter Tuning and Training                                     | 17        |
| 4.3.1    | Hyperparameter Search                                                           | 17        |
| 4.3.2    | Training and Validation                                                         | 18        |

|            |                                                                 |           |
|------------|-----------------------------------------------------------------|-----------|
| <b>4.4</b> | <b>LSTM: Hyperparameter Tuning and Training .....</b>           | <b>18</b> |
| 4.4.1      | Scaling and Normalisation.....                                  | 18        |
| 4.4.2      | Hyperparameter Search .....                                     | 18        |
| 4.4.3      | Training and Validation .....                                   | 19        |
| <b>4.5</b> | <b>Summary of Final Configurations.....</b>                     | <b>19</b> |
| <b>5</b>   | <b><i>Results and Evaluation</i> .....</b>                      | <b>20</b> |
| <b>5.1</b> | <b>Test Set Results .....</b>                                   | <b>20</b> |
| <b>5.2</b> | <b>Validation vs Test Performance.....</b>                      | <b>20</b> |
| <b>5.3</b> | <b>Residual and Bias Analysis .....</b>                         | <b>20</b> |
| <b>5.4</b> | <b>Explainability (SHAP) .....</b>                              | <b>22</b> |
| <b>5.5</b> | <b>Computational Cost Comparison .....</b>                      | <b>23</b> |
| <b>6</b>   | <b><i>Model Compression</i> .....</b>                           | <b>23</b> |
| <b>6.1</b> | <b>Compression Methodology .....</b>                            | <b>23</b> |
| <b>6.2</b> | <b>XGBoost Compression Results .....</b>                        | <b>24</b> |
| <b>6.3</b> | <b>LSTM Compression Results .....</b>                           | <b>25</b> |
| <b>6.4</b> | <b>Cross-Model Discussion .....</b>                             | <b>26</b> |
| <b>7</b>   | <b><i>Discussion, Conclusion, and Recommendations</i> .....</b> | <b>26</b> |
| <b>7.1</b> | <b>Summary of Key Findings .....</b>                            | <b>26</b> |
| <b>7.2</b> | <b>Sustainability Discussion.....</b>                           | <b>27</b> |
| <b>7.3</b> | <b>Limitations and Real-World Application .....</b>             | <b>27</b> |
| 7.3.1      | Limitations .....                                               | 27        |
| 7.3.2      | Real-World Application .....                                    | 27        |
| <b>7.4</b> | <b>Recommendations for Future Work and Conclusion.....</b>      | <b>28</b> |
| 7.4.1      | Future Work .....                                               | 28        |
| 7.4.2      | Conclusion .....                                                | 28        |
|            | <b><i>References</i>.....</b>                                   | <b>29</b> |
|            | <b><i>Appendices</i>.....</b>                                   | <b>30</b> |

## 1 Introduction

### 1.1 Sustainability Challenge and Motivation

Electricity demand forecasting is important for a sustainable, low-carbon energy system. A reliable forecast helps grid operators plan supply accurately.

In the United Kingdom, the electricity sources have been transitioning toward renewable generation for years. In 2025, around 44% of electricity in Great Britain was generated from renewable sources, such as wind and solar (National Energy System Operator (NESO), 2026). But wind and solar output is intermittent, so accurate short-term demand forecasts are needed to manage this. Short-term load forecasting (STLF) supports many grid decisions, from scheduling generation to shifting demand, and helps keep supply reliable (Pinheiro, Madeira and Francisco, 2023).

Better forecasting reduces the need for backup (reserve) generation. This supports UN Sustainable Development Goal 7 (Affordable and Clean Energy) and Goal 13 (Climate Action) (United Nations General Assembly, 2015)

AI and machine learning also raise their own sustainability concerns: training and running large models uses computational resources and energy (Khan *et al.*, 2026). Any deployed model should not add unnecessary load to the grid. For this reason, Section 6 also evaluates model efficiency and compression as a sustainability measure.

### 1.2 Problem Context

Electricity demand is shaped by many factors: weather, seasons, human activity, economic activity, and disruptive events such as COVID-19. A **short-term daily forecast** needs to capture both fast-moving daily changes and slower structural trends.

This project uses four predictor categories: **weather**, **calendar effects (to capture human activity)**, **economic indicators (PMI and RSI)**, and **COVID-19 impact**.

A **daily forecast target** was chosen partly to accommodate PMI and RSI, which are only published monthly. Each month's value is carried across every day in that month, so these indicators can still be used as predictors. **Daily granularity** also matches how GB electricity markets typically schedule generation, one day ahead.

### 1.3 Aims and Objectives

#### Project aims:

1. Develop and compare machine learning and deep learning approaches for short-term forecasting of UK electricity demand, evaluating predictors spanning weather, calendar effects, economic activity, and COVID-19-related disruption.
2. Investigate model interpretability and computational efficiency as sustainability-relevant dimensions of the forecasting pipeline, alongside predictive accuracy.

#### Objectives:

- Build a cleaned, merged dataset spanning electricity demand (2009–2024), weather, calendar, economic activity, and COVID-19 stringency, from multiple open data sources.
- Conduct exploratory data analysis (EDA) to characterise the demand series and justify each predictor category.
- Train and evaluate two supervised learning approaches — a tree-based ensemble (XGBoost) and a deep learning sequence model (LSTM).

- Apply an explainable AI technique (SHAP) to interpret each predictor's contribution to model forecasts.
- Apply model compression to both models and evaluate the trade-off between accuracy and efficiency.
- Draw conclusions on the sustainability implications of the approach, and identify limitations and future work.

## 1.4 Data Review

This project uses daily NESO electricity demand for Great Britain (2009–2024), combined with population-weighted weather from 11 regional stations, the OxCGRT COVID-19 stringency index, and economic indicators (PMI, RSI).

After cleaning and merging, the dataset has around 5,600 daily observations across roughly 20 predictor variables, spanning weather, calendar, economic, and COVID-19 categories.

## 1.5 Report Structure

Section 2 reviews the literature behind our methodological choices. Section 3 covers the data, EDA, and feature engineering. Section 4 details the modelling approach. Section 5 presents evaluation results and explainability. Section 6 reports model compression. Section 7 concludes with findings, limitations, and recommendations.

## 2 Literature Review

This section reviews ten papers on forecasting UK electricity demand. It justifies the choices made in this project: which predictors to use, which models to train, how to explain them, and how to compress them.

### 2.1 Factors Influencing Electricity Demand

Electricity demand does not move at random. It is shaped by human activity, economic activity, and weather. The literature is clearest on weather and calendar effects.

**Weather** has the strongest support. Bansal (2026) uses temperature, humidity, and wind speed as core LSTM/Transformer inputs. Bhupatiraju and Ahn (2025) show temperature-based features rank among the most important predictors at peak demand. Pinheiro, Madeira and Francisco (2023) model temperature's effect as non-linear, not straight-line. Vivas, Allende-Cid and Salas (2020), reviewing 257 studies, find weather variables help most beyond a few hours ahead.

**Calendar effects** are also well supported. Anderson and James (2021) show a strong weekday–weekend pattern in UK demand. Bansal (2026) and Bhupatiraju and Ahn (2025) both find day-of-week and holiday indicators highly important. Pinheiro, Madeira and Francisco (2023) treat day-type as a core part of their model.

**Economic activity** has weaker and less UK-specific support. Vivas, Allende-Cid and Salas (2020) note some studies find GDP or income helpful at longer horizons; Dong et al. (2025) mention economic conditions only in passing. **None** of the ten papers **test PMI or RSI** specifically. This project treats economic activity as an **open question** and tests it directly in Section 3.

### 2.2 UK Electricity Demand Under Disruption: The COVID-19 Case

COVID-19 is a well-documented UK disruption to electricity demand. Anderson and James (2021) show demand fell up to 20% during the first 2020 lockdown's morning peak, and around

11% later in the day. Liu and Lin (2021) use a Bi-LSTM model and find transport-linked demand fell by up to 52% under strict lockdown.

Both studies show UK demand is not stationary or purely seasonal. This justifies including a COVID-19 stringency predictor, and supports using an LSTM capable of handling non-stationary dynamics.

### 2.3 Forecasting Model Approaches for Electricity Demand

No single model wins outright; the best choice depends on the data and features used.

Vivas, Allende-Cid and Salas (2020) find machine learning and hybrid methods generally beat statistical methods like ARIMA. Dong et al. (2025) confirm **deep learning** is now dominant, though **tree-based** methods remain competitive.

Three model families recur: **XGBoost (tree-based)**, used by Pinheiro, Madeira and Francisco, 2023; Bhupatiraju and Ahn, 2025), **LSTM (recurrent networks with memory)**, used by Liu and Lin, 2021; Bhupatiraju and Ahn, 2025; and Bansal, 2026), and **Transformers (attention-based)**, used by Bansal, 2026).

Bhupatiraju and Ahn (2025) compare all of these directly. With good feature engineering, tuned XGBoost (MAPE 0.79%) beat both LSTM (1.21%) and Bi-LSTM (1.14%). This shows deep learning is not automatically better. Pinheiro et al. (2023) found their tree-based model matched a simpler model's accuracy (RMSE 199 MW vs 191 MW) but needed more tuning and was harder to interpret.

Bansal (2026) compares LSTM and Transformers by horizon: LSTM wins at one hour ahead (2.55% MAPE), but the **Transformer** edges ahead at one day ahead (6.88% vs 7.15%) – only a **small gap**. Given this small, horizon-dependent gain, **this project uses LSTM**, as it is simpler, more established, and less data-hungry, matching this project's sustainability aim of avoiding unnecessary computational cost.

Together, this justifies training and comparing two models — **XGBoost** and **LSTM** — with Section 5 deciding which fits this problem best.

### 2.4 Explainability in Load Forecasting

Pinheiro et al. (2023) show that accuracy can come at the cost of transparency. This matters as the grid operators need to understand why a model makes a forecast, not just whether it's accurate.

Both explainability papers reviewed (Lee *et al.*, 2023; Bhupatiraju and Ahn, 2025) use **SHAP**, not alternatives like LIME. Lee, Oh, Kim and Kim (2023) apply it to XGBoost on Korean grid data; Bhupatiraju and Ahn (2025) apply it across five model types. SHAP's game-theory foundation gives it a **more rigorous, consistent basis** than approximation-based methods like LIME — this is why the project adopts it.

Bhupatiraju and Ahn (2025) also show **SHAP** works differently by model type: **tree-based** models allow an **exact calculation**, while **deep learning** models need an **approximation-based explainer**. This project follows the same split — a tree-specific explainer for XGBoost, an approximation-based one for LSTM (detailed in Section 5).

### 2.5 Sustainable AI: Model Compression and Efficiency

Training and running ML models has a real energy cost (Khan *et al.*, 2026). Model compression shrinks a trained model and speeds it up, while trying to keep accuracy close to the original.

Khan et al. (2026) identify three main techniques: **pruning** (removing low-value parameters), **quantization** (lower numerical precision, e.g. 8-bit instead of 32-bit — Paula et al., 2025, report 21–54% inference energy savings this way), and **distillation** (training a smaller "student" model from a larger "teacher," as in DistilBERT).

Paula et al. (2025) test these techniques, alone and combined, on four transformer models. Results vary by model: combining pruning and distillation cut energy by 32.1% for BERT and 23.9% for ELECTRA, both with over 95% accuracy retained.

But pruning an already-compact model (DistilBERT) increased energy use by 6.7%, and quantizing ALBERT saved only 7.1% while costing significant accuracy. **The lesson:** a technique's **benefit depends** on how much redundancy the model already has — what works for a large model can backfire on a small one.

This directly informs **this project's** own approach: **applying individual and combined compression** techniques to both **XGBoost** and **LSTM**, and reporting the accuracy/efficiency trade-offs in Section 6.

## 2.6 Summary and Positioning of This Study

Among all the reviewed literature, three points stand out. First, weather and calendar effects are well supported, but **economic indicators** are not — **this project tests** that **directly** rather than **assuming it**. Second, no single model wins outright, so this project trains and compares both XGBoost and LSTM instead of picking one in advance. Third, compression (Paula *et al.*, 2025; Khan *et al.*, 2026) has only been tested on general-purpose language models, and none of the forecasting papers reviewed test compression at all.

This leaves a gap: **no reviewed study** combines UK electricity demand forecasting, a **direct XGBoost-vs-LSTM comparison**, SHAP explainability for both, and compression testing. This project brings these **four elements together**, treating sustainability as both the motivation for the work and a criterion the final models are judged against.

## 3 Data Understanding, Pre-processing, and Exploratory Data Analysis

This section describes the data and how it was prepared for modelling. It covers the data sources (3.1–3.2), an exploratory analysis of each predictor category from Section 2 (3.3), checks for redundancy between predictors (3.4), feature engineering (3.5), and the data split used for modelling (3.6–3.7).

### 3.1 Data Sources and Data Dictionary

This project combines five data sources into one daily dataset for Great Britain, from 1 January 2009 to 11 June 2024 (5,641 daily rows before feature engineering).

#### 3.1.1 Data sources:

**Electricity demand** — daily national demand from NESO (National Energy System Operator). This is the target variable.

**Weather** — originally from MET Office, recorded every 30 minutes at individual stations. This project uses 11 regional stations (Appendix Table 3), combined into one national daily figure using population weighting (Appendix Table 2) (Section 3.2).

**Economic activity** — PMI and RSI, both published monthly.

**COVID-19 response** — the OxCGRT Stringency Index, recorded daily.

**Calendar** — a UK public holiday flag, merged in from an external source. Other calendar features (day of week, month, weekend flag) are derived from the date itself, and added during exploration in Section 3.3.

(Remark: all data source links and file names are in Appendix Table 1)

### 3.1.2 Data Dictionary (merged dataset, before feature engineering):

Table 1: Data Dictionary (Merged Dataset)

| Field                   | Description                                         | Unit / Type  | Native Frequency                                 |
|-------------------------|-----------------------------------------------------|--------------|--------------------------------------------------|
| Date                    | Calendar date                                       | Date         | Daily                                            |
| nd_total_mwh            | Total GB electricity demand (Target Variable)       | MWh          | Daily (Originally half-hourly, aggregated daily) |
| is_holiday              | UK public holiday flag                              | Binary (0/1) | Daily                                            |
| min_temp                | Minimum temperature, national (population-weighted) | °C           | Originally half-hourly, 11 stations              |
| max_temp                | Maximum temperature, national (population-weighted) | °C           | Originally half-hourly, 11 stations              |
| Rain                    | Rainfall, national (population-weighted)            | mm           | Originally half-hourly, 11 stations              |
| Humidity                | Relative humidity, national (population-weighted)   | %            | Originally half-hourly, 11 stations              |
| cloud_cover             | Cloud cover, national (population-weighted)         | %            | Originally half-hourly, 11 stations              |
| wind_speed              | Wind speed, national (population-weighted)          | km/h         | Originally half-hourly, 11 stations              |
| Pmi                     | Purchasing Managers' Index                          | Index value  | Monthly                                          |
| rsi_volume              | Retail Sales Index, sales volume                    | Index value  | Monthly                                          |
| rsi_mom_change          | Retail Sales Index, month-on-month % change         | %            | Monthly                                          |
| StringencyIndex_Average | OxCGRT COVID-19 stringency index                    | 0-100 scale  | Daily                                            |

(Note: the raw PMI dataset only has PMI value, while the raw RSI dataset has both *rsi\_volume* and *rsi\_mom\_change*, since ONS publishes RSI's month-on-month change directly, whereas the PMI source does not.)

### 3.2 Data Preparation and Merging

Each data source has a different native frequency (Table 1). The first task was converting them all to one daily row before exploration could begin.

#### Aggregating to daily:

- **Electricity demand** — originally recorded every 30 minutes ("settlement periods"). Each day's demand is the sum of that day's settlement-period readings, converted from average power to total energy (MWh).
- **Weather** — originally recorded every 30 minutes. 11 selected regional stations were selected to represent regions (Appendix Table 3). Each station is weighted by its region's 2021 population (Appendix Table 2), so the figure reflects conditions where people — and demand — actually are. Readings were first aggregated to a daily figure per station, then combined into one national daily figure.
- **Economic indicators (PMI, RSI)** — published monthly. Rather than drop them, each month's value is carried across every day in that month (a "step function"). This lets them sit alongside the daily predictors.
- **COVID-19 stringency index** — already daily, so no aggregation was needed. It was merged directly by date. Days outside OxCGRT's tracked window (before 2020 or after 2022) were set to 0, since no pandemic restrictions applied then.



### Merging and trimming:

- All sources were joined on date (or year-month, for PMI/RSI).
- The dataset was trimmed to the date range common to all sources — bounded by the weather data, which ends earliest (11 June 2024).
- Result: **5,641 daily rows, 1 January 2009 to 11 June 2024, 13 columns**, no missing values (**no missingness**), no duplicate dates.

## 3.3 Exploratory Analysis of Demand Drivers

We first establish the demand series' own baseline behaviour — its distribution and structure as a time series in 3.3.1. The remaining subsections test each candidate driver from the Literature Review: calendar effects (3.3.2), weather (3.3.3), economic indicators (3.3.4), and COVID-19 (3.3.5).

### 3.3.1 Understanding the Variables

Daily demand averages 753,089 MWh (std 133,761 MWh), with a roughly symmetric, bell-shaped distribution across the full period.

The full period (2009-2024) demand was decomposed using **STL (Seasonal-Trend decomposition using LOESS)**, with a 365-day period to capture the annual cycle. (Figure 1)

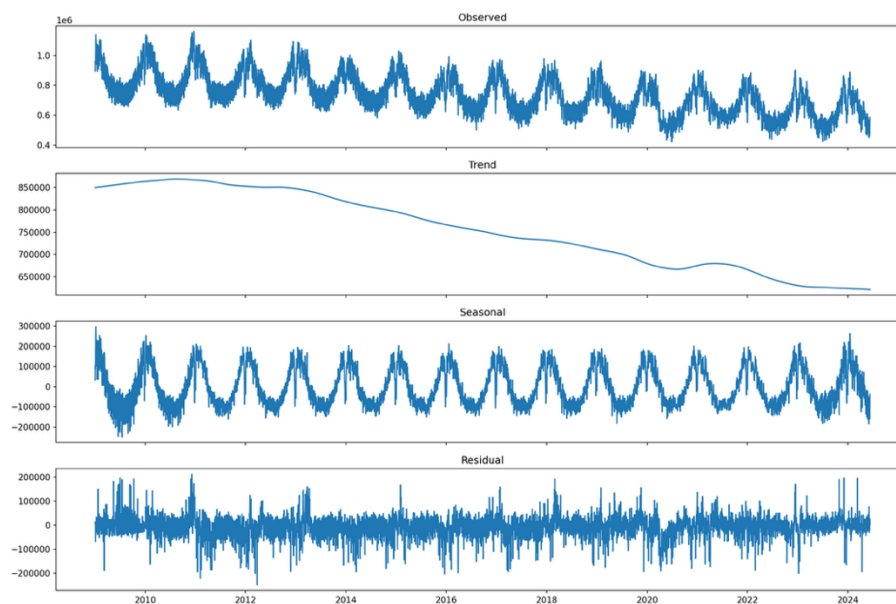


Figure 1: STL Decomposition

This shows the observed series alongside its three components.

**The trend component** declines gently from around 850,000 MWh in 2009 to around 620,000 MWh by 2024, consistent with well-documented long-term efficiency improvements in UK electricity use.

**The seasonal component** shows a consistent annual pattern.

**The residual component** is notably small and stable for most of the series, but visibly widens during 2020 — an early indication of the COVID-19 disruption examined in Section 3.3.5.

An Augmented Dickey-Fuller (ADF) test on the raw series gave a statistic of -3.4597 ( $p=0.0091$ ), which technically indicates stationarity. This needs a caveat: the ADF test only

checks for a unit root, not for trend or seasonality — and the STL decomposition above clearly shows both are present. A low p-value here means "no unit root," not "no trend."

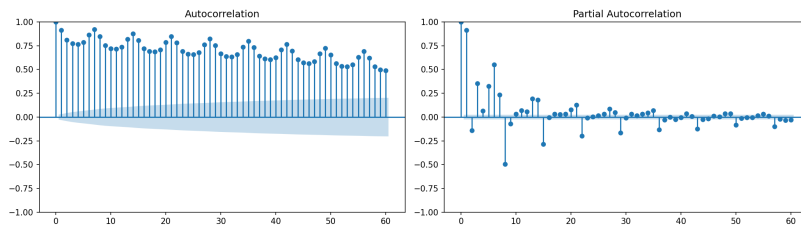


Figure 2: ACF and PACF test. (Left - ACF; Right - PACF)

The autocorrelation (ACF) and partial autocorrelation (PACF) plots (Figure 2), out to 60 lags, show strong and slowly-decaying autocorrelation typical of a trending, seasonal series, with a distinct recurring spike at lag 7 reflecting the weekly cycle already visible in the raw time series.

This pattern directly motivates the lag- and rolling-window features engineered in Section 3.5 (demand\_lag\_1, demand\_lag\_7, and the 7- and 30-day rolling means), which later prove to be among the strongest predictors in the final feature set.

### 3.3.2 Calendar Effects

Electricity demand follows a clear weekly pattern. Average demand is 780,549 MWh on weekdays, compared to 684,454 MWh on weekends — a 12.3% drop, highly significant (Welch's  $t=26.80$ ,  $p<0.001$ ). This matches Anderson and James (2021), and fits with less workplace, school, and commercial activity at weekends.

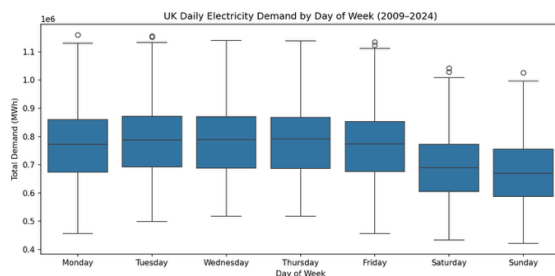


Figure 3: Daily Demand by Day of Week (2009-2024)

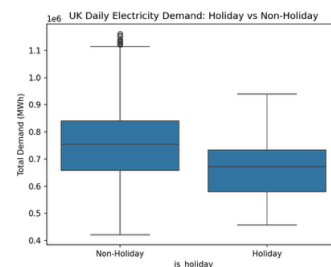


Figure 4: Daily Demand: Holiday vs Non-Holiday

Figure 3 shows this directly: Monday to Friday sit at a similar, higher level, then drop for Saturday and Sunday.

UK public holidays show a similar effect, but measuring it needs care. A naive comparison — holidays against all non-holidays, including weekends — shows an 11.6% drop ( $t=-9.43$ ,  $p<0.001$ ). But this is misleading: UK holidays always fall on a weekday, so this comparison partly just re-captures the weekend effect above, not the holiday effect itself. Figure 4 reflects this same naive comparison.

Comparing holiday weekdays against ordinary weekdays only gives the true holiday effect: a 15.0% drop ( $t=-12.50$ ,  $p<0.001$ ) — larger than the naive figure suggested.

This matters for feature engineering (Section 3.5): a holiday's effect is not fully explained by it falling on a weekday, so the model needs a weekend indicator and a holiday indicator as separate features.

Both **is\_weekend** and **is\_holiday** are kept as **separate predictors**, since each captures a distinct, significant drop in demand not fully explained by the other.

### 3.3.3 Weather Relationships

Temperature has the strongest raw relationship with demand: max temperature correlates at  $r=-0.649$ , min temperature at  $r=-0.628$ , both strongly significant. Humidity shows a moderate positive correlation ( $r=0.547$ ). Rain, wind speed, and cloud cover are all weakly related ( $|r|<0.08$ ).

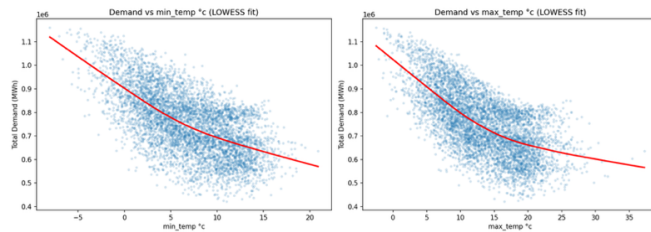


Figure 5: LOWESS Scatter - Demand vs min temp (left) and max temp (right)

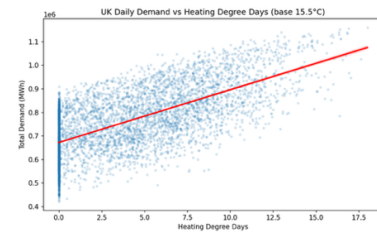


Figure 6: Scatter Plot - Demand vs HDD

Figure 5 shows the relationship is not a straight line: demand falls steeply as temperature rises from cold winter values, then flattens out at milder and warmer temperatures.

To capture this directly, **Heating Degree Days (HDD, base 15.5°C)** were calculated — a standard measure of how far a day's temperature falls below the point where heating is typically needed. Days above this threshold contribute zero.

HDD correlates with demand at  $r=0.666$ , stronger than raw temperature ( $r=-0.649$ ). This confirms a heating-driven, threshold-based relationship fits better than a simple linear one.

Figure 6 shows this: a visibly cleaner, more linear relationship, supporting HDD's use as an engineered feature in Section 3.5.

The temperature effect also varies by season: winter ( $r=-0.561$ , slope -22,187 MWh/°C), spring ( $r=-0.491$ ), and autumn ( $r=-0.557$ ) all show a strong, similar effect. Summer is much weaker ( $r=-0.136$ , slope -4,239 MWh/°C) — around a fifth of winter's slope.

**Humidity's** correlation could simply reflect its own link to temperature (humid days are often cold days). A partial correlation, controlling for max temperature, checks this:  $r=0.407$ ,  $p<0.001$  — still significant. So humidity carries information beyond temperature alone, not just a proxy for it.

**Rain, wind speed, and cloud cover** all show similarly **weak relationships** with demand (see Appendix Figure 6).

The intercorrelation matrix (Figure 7) shows some correlation between humidity and cloud cover, revisited in Section 3.4's cross-predictor check. No pair is strong enough to suggest the weather variables simply duplicate each other.

On this evidence, **HDD** is retained as the **primary temperature predictor** — stronger, more clearly heating-driven relationship with demand. **Humidity** is retained given its **confirmed independent contribution**. **Rain, wind speed, and cloud cover**

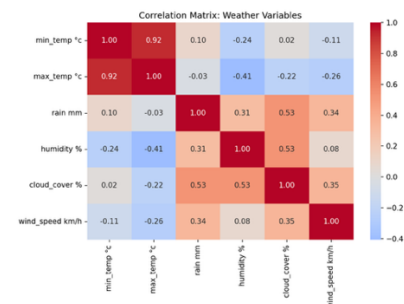


Figure 7: Intercorrelation Matrix among Weather Variables

are carried forward as **provisional candidates**, to be confirmed by the cross-predictor check in Section 3.4.

### 3.3.4 Economic Indicators

Section 2.1 found only thin, non-UK-specific literature support for economic indicators. This subsection tests PMI and RSI directly against the UK data.



Figure 8: STL Daily Demand Trend, PMI Trend and RSI Trend

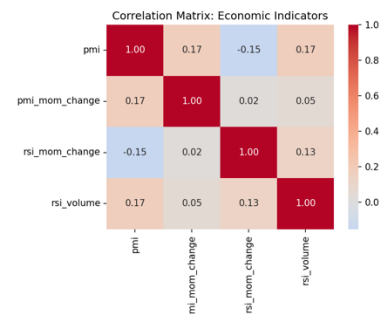


Figure 9: Correlation Matrix - PMI and RSI variables

Figure 8 plots the demand trend against PMI and RSI. PMI shows no obvious link to the declining demand trend. RSI looks strikingly similar: both series rise steadily until around 2020, then follow a similar disrupted path through the pandemic.

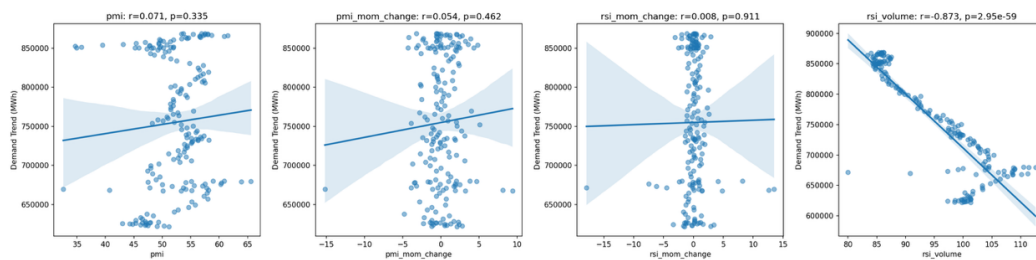


Figure 10: Scatter Plots - Demand vs pmi, pmi\_mom\_change, rsi\_mom\_change and rsi\_volume

The scatter plots (Figure 10) confirm this numerically. PMI's raw correlation with demand is weak and not statistically significant ( $r=0.071$ ,  $p=0.335$ ). Its month-on-month change (pmi\_mom\_change) is similarly weak ( $r=0.054$ ,  $p=0.462$ ) — no meaningful relationship exists for PMI at either resolution. A lag search was also run, checking whether PMI might lead demand at a delay rather than move with it directly; the strongest result was a modest 7-month lag ( $r=0.199$ ,  $p=0.00762$ ) — weak grounds for inclusion on its own, and treated cautiously since testing many lags raises the risk of a chance result (see Appendix Figure 7).

RSI's raw correlation, by contrast, is strikingly strong ( $r=-0.873$ ,  $p<0.001$ ) — strong enough to demand a closer look before accepting it as genuine. Two series can look strongly correlated simply because both trend in the same direction over time, without any real short-run relationship. This is exactly the risk here, since both RSI and demand trend downward across the same period.

To test this, both series were differenced to monthly changes, removing the shared trend. The correlation collapsed to  $r=0.015$  ( $p=0.836$ ) — confirming the raw relationship was spurious. This is corroborated by rsi\_mom\_change, ONS's own independently published month-on-month figure, which also shows a negligible correlation ( $r=0.008$ ,  $p=0.911$ ). The four economic

indicators also show only weak correlation with each other (all  $|r| \leq 0.17$ , Figure 9), so none is a redundant duplicate of another.

On this evidence, **pmi**, **pmi\_mom\_change**, **rsi\_volume**, and **rsi\_mom\_change** are **all excluded** from the feature set — confirmed independently by the VIF check in Section 3.4.

This **doesn't rule out economic activity influencing demand** more broadly; that possibility is revisited in Section 7 as future work.

### 3.3.5 COVID-19 Disruption

Section 2.2 identified COVID-19 as a genuine, well-documented disruption to UK demand. This subsection tests it directly, using the OxCGRT stringency index as a daily measure of restriction severity, from 1 January 2020 to 31 December 2022 (1,096 days, 1,077 with a stringency value above zero).

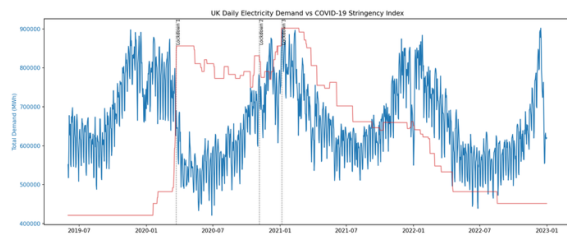


Figure 11: Daily Demand and COVID-19 Stringency Index across 2020–2022

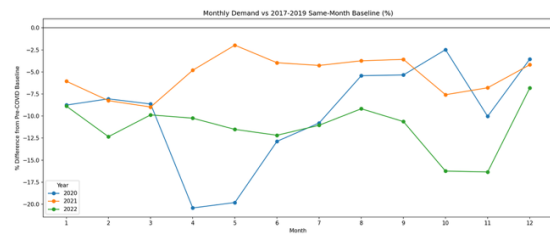


Figure 12: Covid Years (2020–2022) Monthly Demand vs (2017–2019) Same-Month Baseline (%)

Figure 11 plots demand and stringency together, with the three UK lockdown start dates marked. Demand visibly drops at each lockdown, most sharply at the first.

A naive correlation between demand and stringency is weak and not significant ( $r=0.040$ ,  $p=0.187$ ) — and even points the wrong way. This is a confound: the first and third lockdowns both fell in cold months, when heating demand is naturally high, hiding any COVID-specific drop. Controlling for this with a partial correlation against HDD gives the expected result:  $r=-0.073$  ( $p=0.016$ ) — modest, but correctly signed and significant.

Figure 12 gives a cleaner test: each COVID-era month compared against the same month's average in the three pre-COVID years (2017–2019). Every month from January 2020 to December 2022 shows a drop, from about -2% to -20%, deepest during the first lockdown (April 2020: -20.4%; May 2020: -19.8%). This drop does not fade by 2022: several late-2022 months (October: -16.2%; November: -16.3%) show reductions as deep as the original lockdowns — a sustained shift, not a temporary one.

The cleanest single comparison — first-lockdown weekdays (23 March–15 June 2020) against the same weekdays in the three pre-COVID years — gives an unconfounded estimate: demand fell from 695,505 MWh to 566,283 MWh, an 18.6% drop ( $t=-16.54$ ,  $p<0.001$ ). This matches Anderson and James's (2021) reported morning-peak reduction of up to 20% for the same period (Section 2.2), corroborating this project's own measurement.

On this evidence, the **OxCGRT stringency index** is retained as a **predictor** (Section 3.5): it captures a genuine, corroborated, and persistent effect, not a transient one that could be safely dropped.

### 3.4 Cross-Predictor Checks

Section 3.3 tested each predictor individually against demand. Before finalising the feature set, we also need to check whether any predictors are redundant with each other — two highly correlated predictors add complexity without adding independent information, and can destabilise a model's coefficients. This also resolves the "keep and review" status given to rain, wind speed, and cloud cover in Section 3.3.3.

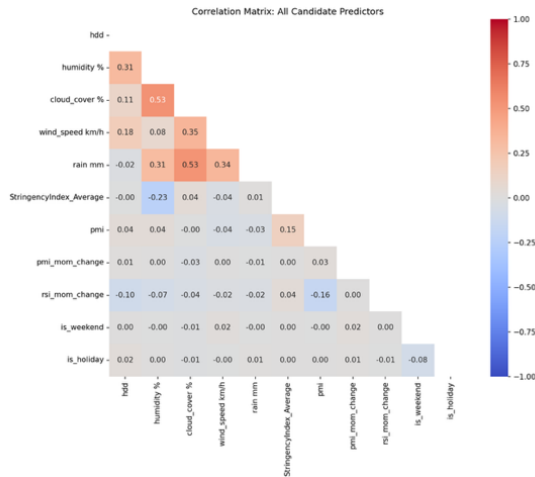


Table 2: Variance Inflation Factors (VIF) of Features

| Feature                 | VIF   |
|-------------------------|-------|
| cloud_cover %           | 1.974 |
| humidity %              | 1.819 |
| rain mm                 | 1.491 |
| wind_speed km/h         | 1.295 |
| hdd                     | 1.218 |
| StringencyIndex_Average | 1.164 |
| pmi                     | 1.065 |
| rsi_mom_change          | 1.041 |
| is_holiday              | 1.009 |
| is_weekend              | 1.008 |
| pmi_mom_change          | 1.003 |

Figure 13: Correlation Matrix for all eleven candidate predictors

Figure 13 shows the correlation matrix across all eleven remaining predictors (weather variables, HDD, COVID stringency, the two calendar flags, and — for completeness — PMI and RSI's month-on-month change, despite their exclusion in Section 3.3.4). The only notable correlation is between humidity and cloud cover ( $r=0.532$ ,  $p<0.001$ ) — humid days tend to be cloudier, which is unsurprising. No other pair exceeds 0.3.

To check whether this is a genuine concern, Variance Inflation Factors (VIF) were computed for all eleven predictors (Table 2). All values sit well below both the moderate-concern threshold ( $VIF>5$ ) and the serious-concern threshold ( $VIF>10$ ) — including cloud cover's, despite its correlation with humidity. Rain, wind speed, and cloud cover can all proceed into Section 3.5 without concern; their weaker individual relevance (Section 3.3.3) means their contribution may still be modest, but multicollinearity is not an issue.

A further question: does HDD simply duplicate the calendar month feature used in Section 3.5? Two ANOVA tests check this. HDD varies significantly by month ( $F=1235.04$ ,  $p<0.001$ ) — expected, since heating demand is seasonal — but not by day of week ( $F=0.08$ ,  $p=0.998$ ). The month result doesn't make HDD redundant, though: month only captures a fixed seasonal average, while HDD captures the actual temperature on a given day — exactly the extra, non-seasonal signal a forecasting model needs.

On this evidence, **no candidate predictor is dropped** on multicollinearity grounds. **Rain, wind speed, and cloud cover are confirmed safe to retain.** PMI, pmi\_mom\_change, rsi\_volume, and rsi\_mom\_change remain excluded, per the relevance-based decision already made in Section 3.3.4 — this section's VIF check is consistent with that decision, not the basis for it.

### 3.5 Feature Engineering

With the candidate predictors confirmed in Sections 3.3–3.4, this section builds the final model-ready dataset.



**Cyclical encoding.** Calendar month and day of week were originally coded as plain integers (1–12, 0–6). This is a problem: a raw integer implies December (12) and January (1) are eleven units apart, when they're actually adjacent. Both were re-coded as a sine/cosine pair (month\_sin/month\_cos, dow\_sin/dow\_cos), which correctly represents their cyclical nature.

**Lag and rolling-window features.** Motivated by the strong autocorrelation and weekly spike seen in Section 3.3.1, five features capture demand's recent history: the previous day (demand\_lag\_1), the same day one week prior (demand\_lag\_7), 7- and 30-day rolling means, and a 7-day rolling standard deviation (all using only prior days, to avoid leaking same-day information).

**Interaction term.** Sections 3.3.2 and 3.3.3 showed the weekday/weekend effect and the HDD effect are both large and independent. An interaction term, hdd\_x\_weekend, lets the model capture whether the heating response itself differs between working and non-working days.

**Feature exclusions.** STL's decomposition components (trend, seasonal, resid) are, by construction, derived directly from demand itself, so none were carried into the feature set — a standard precaution against target leakage. The feature, year, was also excluded, to prevent the model from directly memorising the historical decline rather than learning it from HDD, calendar, and stringency features. Also, is\_covid\_tracked, a window-marker redundant with the retained StringencyIndex\_Average, was dropped as redundant.

**Final dataset.** After these steps, the dataset has **21 columns**. Removing the first 30 rows (which have **missing values** from the lag/rolling calculations) leaves 5,611 rows, saved as model\_ready\_dataset.csv.

As expected from Section 3.3.1, the lag and rolling-window features are the strongest predictors in the dataset: demand\_lag\_7 ( $r=0.923$ ), demand\_lag\_1 ( $r=0.913$ ), and the 7-day rolling mean ( $r=0.903$ ) all lead the full ranking (see Appendix Table 4 for correlations across all 20 features).

### 3.6 Train/Validation/Test Split

This is a time series, not a typical dataset — rows cannot be shuffled into random splits, since that would let a model be evaluated on data from before some of its own training data (look-ahead bias). Instead, the split follows calendar years (Table 3), so each split contains at least one full annual cycle — important given the strong seasonality confirmed in Section 3.3.1.

Table 3: Train/Validation/Test Split

| Split      | Date range               | Rows  | % of total |
|------------|--------------------------|-------|------------|
| Train      | 2009-01-31 to 2021-12-31 | 4,718 | 84.1%      |
| Validation | 2022-01-01 to 2022-12-31 | 365   | 6.5%       |
| Test       | 2023-01-01 to 2024-06-11 | 528   | 9.4%       |

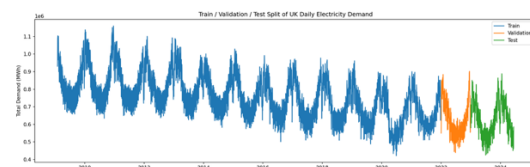


Figure 14: Full Demand Series with Train/Validation/Test Split

(Training starts on 31 January, not 1 January 2009, since the first 30 rows were removed in Section 3.5 for the lag and rolling-window calculations.)

Figure 14 shows the full series with each split colour-coded, confirming no gaps or overlaps.

One thing is worth noting: the validation set (2022) is the last year inside the OxCGRT COVID window. The entire test set (2023 onward) falls after it, with the stringency index at zero

throughout. So the test set evaluates the model in a period the COVID predictor cannot distinguish from any other normal period — revisited in Section 5.

### 3.7 Scaling and Normalisation

**Feature scaling** is **not applied yet**, for a reason specific to each model. **XGBoost** splits on raw feature thresholds, so scaling has **no effect** on it. **LSTM**, however, is **sensitive to input scale** during training. Its features and target are standardised separately in **Section 4**, using a scaler fit only on the training data — to avoid leaking validation or test information into that scaling.

## 4 Modelling

### 4.1 Problem Formulation

This project frames electricity demand forecasting as **supervised regression: predicting** total daily GB demand (`nd_total_mwh`, in MWh) **one day ahead**, using the 20 predictors from Section 3 — weather, calendar, COVID-19 stringency, and the target's own recent history (lag and rolling-window features).

The weather data used here is the **actual observed weather** for the forecast day, not a weather forecast. In a **real deployment**, tomorrow's weather isn't known yet — a **forecast** would be used instead, and any error in that forecast would carry into the demand prediction. The issue will be further discussed in Section 7.

The **same chronological train/validation/test split** (Section 3.6) and the **feature set** (Section 3.5, `model_ready_dataset.csv`) apply to both models. Both models are tuned on the **same validation year (2022)** and **tested** on the **same held-out period** (2023 onward), for a fair comparison.

However, the two models need the same data in **different shapes**. XGBoost takes the data as flat rows — each day is one observation, with lag and rolling features already as columns. LSTM needs the data reshaped into overlapping windows of consecutive days, so it can learn patterns from the sequence itself. This reshaping, and the feature/target scaling deferred from Section 3.7, are described in Section 4.4.

Both models are tuned using validation-set MAE as the primary metric. Full evaluation — MAPE, RMSE, and qualitative analysis — is in Section 5.

### 4.2 Model Selection Rationale

Section 2.3 found no single architecture wins outright at short-term load forecasting — tree-based ensembles and deep learning models each have genuine strengths, depending on feature engineering and forecast horizon. This project picks one of each — XGBoost and LSTM — for direct comparison, rather than assuming either is better in advance.

**XGBoost** was chosen over alternatives like Random Forest, following Bhupatiraju and Ahn (2025), who found gradient-boosted ensembles beat simpler methods once paired with good feature engineering — the same approach this project takes with HDD, lag, and interaction features (Section 3).

**LSTM** was chosen over alternatives like GRU, following Liu and Lin's (2021) use of a Bi-LSTM for UK demand through COVID-19 (Section 2.2), and Section 2.3's finding that LSTM balances sequence-modelling ability and complexity well at a day-ahead horizon, without a



Transformer's extra data and compute needs. LSTM's larger parameter count than GRU also gives more room to show a real before/after effect when compression is applied in Section 6.

Both models are trained from scratch on the same feature set and data split (Section 4.1), so the result in Section 5 — not the literature alone — decides which fits this problem best.

**Strengths and weaknesses.** The two architectures involve a real trade-off, relevant to this project's sustainability framing:

Table 4: Comparison of XGBoost and LSTM

| Dimension                            | XGBoost                                                                      | LSTM                                                                            |
|--------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Training compute cost                | Very low — sub-second training on this dataset                               | Substantially higher — multiple training epochs, GPU-accelerated                |
| Representation of temporal structure | Relies entirely on the hand-engineered lag/rolling features from Section 3.5 | Learns temporal dependencies directly from the input sequence                   |
| Explainability                       | Exact, efficient SHAP calculation (TreeExplainer)                            | Requires an approximation-based explainer (Section 5)                           |
| Data preparation requirements        | No feature scaling needed (Section 3.7)                                      | Requires input scaling and sequence windowing (Section 4.4)                     |
| Model complexity                     | Comparatively small parameter count                                          | An order of magnitude more parameters                                           |
| Tuning stability                     | Deterministic given a fixed seed                                             | Some run-to-run instability under GPU non-determinism (Section 4.4.2 and 4.4.3) |

This **trade-off** between **predictive flexibility** and **computational cost** is revisited in Section 6, where both models are evaluated for compression potential.

### 4.3 XGBoost: Hyperparameter Tuning and Training

#### 4.3.1 Hyperparameter Search

**Baseline diagnostic.** An XGBoost model was first trained with default settings, to check whether tuning was needed. It showed clear overfitting: training MAPE of 0.52% against validation MAPE of 3.78% — a sevenfold gap. The train/validation MAE curve confirms this (Appendix Figure 9): validation error plateaus by round 20–30, while training error keeps falling toward zero by round 500. This pointed toward shallower, more regularised trees.

**Search.** A random search of 150 combinations tested `max_depth`, `eta`, `subsample`, and `colsample_bytree` across a 1,470-combination space (~10% sampled), including the best-known configuration from earlier exploration (`max_depth=3`, `eta=0.3`, `subsample=1.0`, `colsample_bytree=0.7`; validation MAE 18,334) to make sure it wasn't missed.

**The best result:** `max_depth=2`, `eta=0.4`, `subsample=0.8`, `colsample_bytree=0.6`, `n_estimators=122`, giving validation MAE 17,705 (MAPE 2.83%) — an improvement over both the overfit default and the earlier known-good configuration.

**Boundary check.** Since `max_depth=2` sat at the shallow edge of the tested range, `max_depth=1` was also checked; it performed worse (MAE 19,048–20,629), confirming `max_depth=2` as a genuine optimum, not a search-boundary artefact.

The full search took 103.1 seconds. Unlike LSTM (Section 4.4.2), no multi-seed averaging was needed: XGBoost on CPU with a fixed seed is fully deterministic — confirmed when the earlier best-known configuration reproduced its exact result within this search.

### 4.3.2 Training and Validation

Using the final hyperparameters, the model was refit on the combined training and validation data (5,083 rows) before test-set evaluation. This took 0.06 seconds — negligible cost, revisited in Section 6.

The shallow, heavily-regularised final configuration reflects the overfitting diagnosed above: rather than deep, unconstrained trees, the search converged on a simpler model — consistent with a dataset where a few strong predictors (Section 3.5's lag and rolling features) already carry most of the signal. Full test-set evaluation, including comparison against LSTM and the naive baselines, is in Section 5.

## 4.4 LSTM: Hyperparameter Tuning and Training

Unlike XGBoost, LSTM needs its input reshaped from flat daily rows into overlapping sequences. Each sequence spans a 30-day window ending on the forecast day, giving the model that day's weather, calendar, and COVID stringency values — matching what XGBoost's flat row already contains for the same day. This is not target leakage: the lag and rolling-window features use only past values (Section 3.5), and the target itself is excluded from the input entirely; only same-day weather, calendar, and stringency (all known or forecastable in advance) are included.

### 4.4.1 Scaling and Normalisation

Features and target were scaled using a **StandardScaler**, fit **only on training sequences**, then applied to training, validation, and test sequences alike — so no information from validation or test leaks into the scaling.

### 4.4.2 Hyperparameter Search

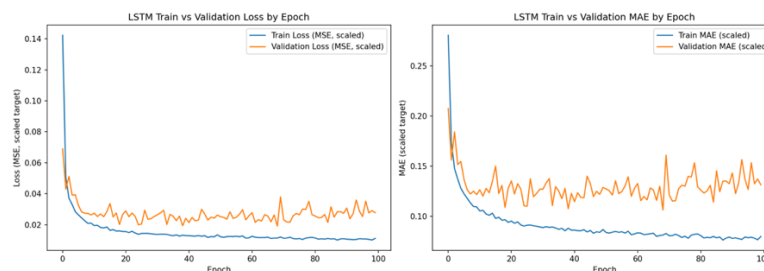


Figure 15: LSTM Train vs Validation (Loss and MAE) by Epoch

**Baseline diagnostic.** A single-layer LSTM (64 units, dropout 0.2, no tuning) was trained for up to 100 epochs without early stopping, to see its natural overfitting pattern (Figure 15). Validation loss was noisy but degraded only gradually — best at epoch 69, only slightly worse by epoch 100. This set the patience (25 epochs) and epoch ceiling (150) used in the search that followed. This is a diagnostic run to set training duration, not a search over epoch count itself.

**Phase 1: random search.** The full search space — `lstm_units`, `dropout`, and `learning_rate` — spans 75 combinations. A random search tested 20 of these (~27%), each with a single seed. The best single-run result: `lstm_units=256`, `dropout=0.5`, `learning_rate=0.001` (MAE=12,913).

**Phase 2: multi-seed confirmation.** LSTM training on GPU is non-deterministic, so a single run can't be trusted alone. The top 3 candidates from Phase 1 were re-run across 3 seeds each (9 further fits) and ranked by mean validation MAE. This changed the winner: `learning_rate=0.001` averaged MAE=13,343 (std=666), while `learning_rate=0.0005` averaged a lower, more stable MAE=13,308 (std=359) — showing why single-run tuning is unreliable here. The scale of this non-determinism is worth stating directly: the same configuration, run

twice with the same seed (42), gave MAE=12,913 once and MAE=13,316 the other time. Same code, same seed, same data, different result.

**The final configuration:** lstm\_units=256, dropout=0.5, learning\_rate=0.0005, median 79 epochs across seeds. Mean validation MAE 13,308 (std=359), MAPE 2.12%. The full two-phase search (29 model fits in total) took 1,966.9 seconds.

#### 4.4.3 Training and Validation

Using this configuration, the model was refit on the combined training and validation sequences (5,054 total: 4,689 training, 365 validation), fixed at 79 epochs with no further early stopping. This took 90.27 seconds — much longer than XGBoost's 0.06 seconds, a cost revisited in Section 6.

**On reproducibility.** Global determinism settings (fixed seeds, TF\_DETERMINISTIC\_OPS, enable\_op\_determinism()) were applied throughout, following best practice for reproducible GPU training. This did not fully guarantee identical results, however: GPU non-determinism appeared in some runs (the baseline model above; the single-seed discrepancy in 4.4.2) but not others — a full re-run of the final configuration reproduced its result exactly. So non-determinism is possible under GPU/cuDNN training even with these safeguards in place, but not guaranteed on every run; both outcomes are reported honestly rather than assuming either always holds.

#### 4.5 Summary of Final Configurations

Table 5 summarises the final configuration, training cost, and validation performance for both models.

Table 5: Summary of Final Configurations of XGBoost and LSTM

|                              | XGBoost                                                                                 | LSTM                                                               |
|------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Final hyperparameters</b> | max_depth=2,<br>eta=0.4,<br>subsample=0.8,<br>colsample_bytree=0.6,<br>n_estimators=122 | lstm_units=256,<br>dropout=0.5, learning_rate=0.0005,<br>79 epochs |
| <b>Tuning time</b>           | 103.1s (1.72 min)                                                                       | 1,966.9s (32.78 min)                                               |
| <b>Final training time</b>   | 0.06s                                                                                   | 90.27s                                                             |
| <b>Validation MAE</b>        | 17,705                                                                                  | 13,308                                                             |
| <b>Validation MAPE</b>       | 2.83%                                                                                   | 2.12%                                                              |

**Tuning time** is a one-off development cost, spent once while searching for a good configuration. **Final training time** is the cost paid every time the model is retrained on new data in deployment. LSTM is far more expensive on both counts — about 19x XGBoost's tuning time, and over 1,500x its final training time — a gap revisited in Section 6's compression analysis. (Note: run times may vary slightly between runs, due to shared hardware in the Colab environment.)

On validation alone, LSTM already beats XGBoost on both MAE and MAPE. Whether this holds on the test set — given the validation-to-test distribution shift noted in Section 3.6 — is addressed fully in Section 5.

## 5 Results and Evaluation

### 5.1 Test Set Results

Both models are now evaluated on the held-out test set (2023 onward) — data neither model saw during training or tuning (Section 3.6). Two naive baselines give context: **Persistence** (repeats yesterday's demand) and **Seasonal-naive** (repeats the same day one year ago).

Table 6: Test Set Results

| Model                  | MAE    | RMSE   | MAPE  | vs. Persistence |
|------------------------|--------|--------|-------|-----------------|
| Persistence (t-1)      | 36,064 | 46,999 | 5.87% | —               |
| Seasonal-naive (t-365) | 50,508 | 64,891 | 8.08% | -40.1%          |
| Tuned XGBoost          | 29,691 | 35,014 | 4.88% | +17.7%          |
| Tuned LSTM             | 14,854 | 19,236 | 2.39% | +58.8%          |

Seasonal-naive performs worse than Persistence: with this much day-to-day structure (Section 3.3.1), yesterday's value is a better reference point than the same day last year. Both trained models comfortably beat both baselines, but the gap between them is large, not marginal: LSTM's MAE is roughly half of XGBoost's, and its MAPE (2.39%) is well under half of XGBoost's (4.88%) — a **decisive win for LSTM**.

This is worth pausing on, since it doesn't follow simply from the validation results in Section 4.5. There, XGBoost and LSTM's validation MAE (17,705 vs. 13,308) suggested a real but more moderate LSTM advantage. Here, on the test set, that advantage more than doubles. Section 5.2 looks at why validation performance didn't predict this test outcome.

### 5.2 Validation vs Test Performance

Validation is meant to reliably predict how a model does on unseen data. This project's results suggest it's less reliable than usually assumed, in two ways.

**The gap between models grew.** On validation, LSTM's MAE (13,308) was 24.8% lower than XGBoost's (17,705). On the test set, that gap grew to 50.0% (14,854 vs. 29,691). LSTM won both times, but the size of its lead wasn't something validation predicted well.

**A ranking flip happened within XGBoost's own tuning.** The earlier known-good configuration flagged during the search (Section 4.3.1) had a worse validation MAE (18,334) than the final selected configuration (17,705). But on the test set, this flipped: the earlier candidate's test MAE (28,316) beat the final configuration's (29,691). The configuration chosen for its better validation score did worse where it actually mattered — clear proof that a lower validation MAE doesn't always mean a lower test MAE.

**Why this happens.** As noted in Section 3.6, the validation year (2022) sits inside the OxCGRT COVID window, while the test set (2023 onward) falls entirely after it. A configuration tuned to fit 2022's leftover pandemic effects doesn't necessarily work as well on a structurally different, post-pandemic period.

This doesn't change Section 5.1's result — **LSTM still clearly wins either way** — but it's a real limitation of validation-based tuning, one that showed up more than once in this project.

### 5.3 Residual and Bias Analysis

Both models over-predict on average: residuals (actual minus predicted) are negative for both, meaning both forecast slightly higher than actual demand. The scale differs a lot: XGBoost's mean residual is -26,967, versus LSTM's -7,418 — XGBoost's bias is about 3.6x larger.

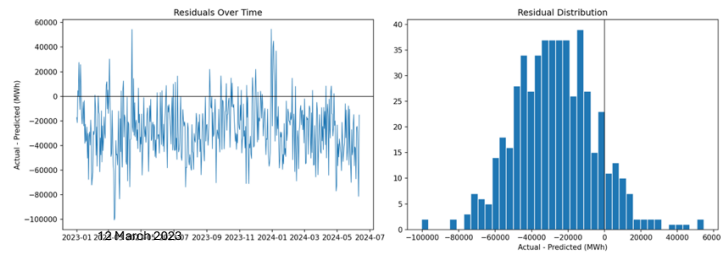


Figure 16: XGBoost Residuals

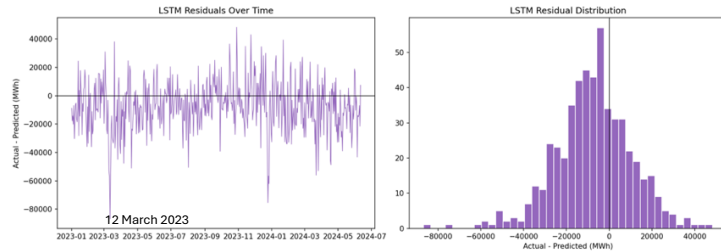


Figure 17: LSTM Residuals

Figure 16 and Figure 17 show each model's residual distribution over the test period.

Table 7: Bias by Day type - XGBoost and LSTM

| Day type       | XGBoost signed bias | LSTM signed bias | XGBoost MAPE | LSTM MAPE    |
|----------------|---------------------|------------------|--------------|--------------|
| Weekday        | -27,418             | -7,241           | 4.76%        | 2.32%        |
| Weekend        | -25,840             | -7,860           | 5.19%        | 2.55%        |
| <b>Holiday</b> | <b>-40,042</b>      | <b>-21,255</b>   | <b>7.49%</b> | <b>4.06%</b> |
| Non-holiday    | -26,610             | -7,041           | 4.81%        | 2.34%        |

Breaking this down by day type (Table 7) shows the same pattern for both models: **holidays are the hardest case**, by a clear margin. Holiday bias is about 1.5x larger than weekday bias for XGBoost, and nearly 3x larger for LSTM. Both models struggle most with the day type that appears least often in training data (Section 3.3.2), where the demand pattern is harder to learn from a smaller sample.

Both models' **single worst over-prediction** falls on the **same day** — **12 March 2023 (a Sunday)** — LSTM misses by 86,757 MWh, XGBoost by 100,461 MWh. Since this is shared across two structurally different models, it points to something genuinely **unusual in that day's real-world demand** — not a model-specific weakness. Neither model's available predictors (weather, calendar, COVID stringency) explain it; this is noted as an open anomaly, not a resolved one, and revisited as a limitation in Section 7.

Since both models show a consistent, not random, bias, a simple fix was tested: subtract a constant offset (the mean residual on the validation set) from every test prediction.

Table 8: Bias correction test

|         | MAE (uncorrected → corrected) | RMSE            | MAPE          |
|---------|-------------------------------|-----------------|---------------|
| XGBoost | 29,691 → 27,371 (-7.8%)       | 35,014 → 32,688 | 4.88% → 4.50% |
| LSTM    | 14,854 → 14,425 (-2.9%)       | 19,236 → 18,777 | 2.39% → 2.31% |

This improved both models, and XGBoost benefits more, matching its larger starting bias. This is a practically useful finding: a near-zero-cost step improves both models' real-world accuracy, with no retraining needed — a low-effort, low-compute fix in line with this project's sustainability aim.

## 5.4 Explainability (SHAP)

Matching the different SHAP approaches for each architecture (Section 2.4), a TreeExplainer was used for XGBoost (exact SHAP values from the tree structure), and a GradientExplainer for LSTM (an approximation, computed on 100 test sequences given the higher cost for deep learning models).

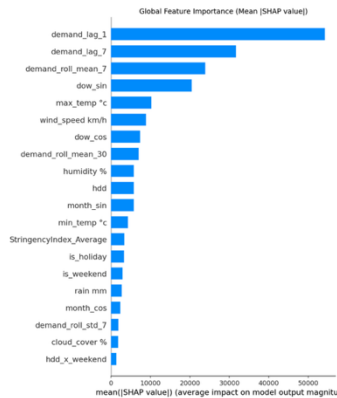


Figure 18: SHAP Feature Importance Ranking - XGBoost

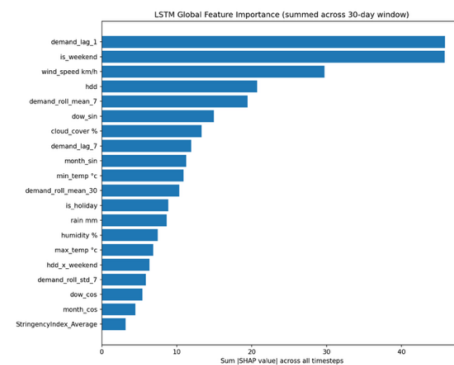


Figure 19: SHAP Feature Importance Ranking - LSTM

For **XGBoost** (Figure 18), **demand\_lag\_1** dominates (mean |SHAP| = 54,267) — about 1.7x the next feature, **demand\_lag\_7** (31,664), followed by **demand\_roll\_mean\_7** (23,912) and **dow\_sin** (20,459). Weather features (max\_temp, wind\_speed, humidity, hdd) rank lower, contributing meaningfully less. This matches the raw correlations from Section 3.5: the features that correlate most with demand are also what the model relies on most.

**LSTM's** ranking (Figure 19) tells a different story. **demand\_lag\_1** is still important (45.8), but **is\_weekend** is essentially **tied with it** (45.8) — a much bigger role than XGBoost gives it. **wind\_speed** also ranks **surprisingly high** for LSTM (29.7, third overall) — above hdd (20.7), the reverse of XGBoost's order.

This is a genuine finding, not a discrepancy to explain away: the **two models** don't just reach similar predictions through different paths — they **weigh** some of the **same evidence differently**. A full explanation is beyond this project's scope, but one plausible reason is that LSTM sees wind speed's pattern across the whole 30-day window, not just a single same-day value like XGBoost does.

Both models were also checked at the single-prediction level. For XGBoost, the forecast for 2 January 2023 (actual 648,132 MWh, predicted 669,513 MWh) can be broken down feature-by-feature (Figure 20), showing exactly what pushed the prediction above baseline.

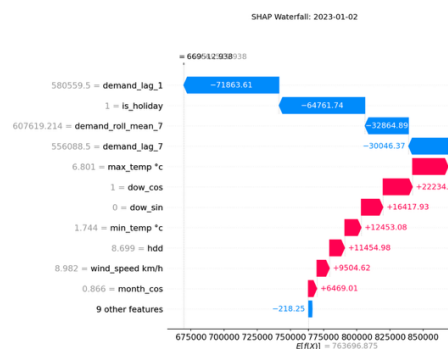


Figure 20: XGBoost – SHAP Waterfall: Single-day decomposition

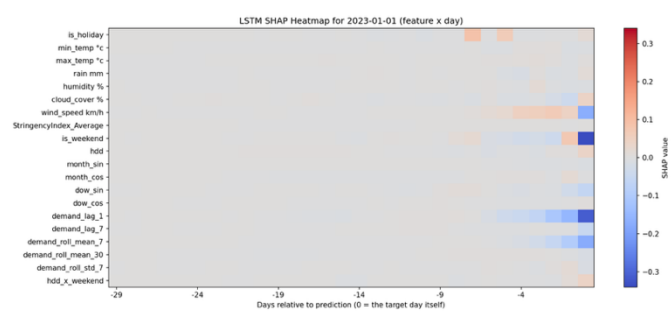


Figure 21: LSTM - SHAP Heatmap, 2023-01-01, decomposition



LSTM can't use the same single-row view, since its input is a 30-day sequence, not one row. Instead, its example (1 January 2023) is a heatmap (Figure 21) showing which features mattered *and* when in the preceding month their influence was strongest.

For **XGBoost**, a local explanation is one number per feature — **simple to read** at a glance. For LSTM, a local explanation is one number per feature *per day* in the 30-day window — so it shows not just what mattered, but when it mattered. That's genuinely richer information, but it also means a single **LSTM** prediction now has **two dimensions to interpret** (feature and time) instead of one, making it inherently **harder to read** than XGBoost's flat, single-dimension view.

This matches the trade-off flagged in Section 4.2 — **XGBoost easier to interpret, LSTM richer but costlier** — a pattern that continues into the computational comparison in Section 5.5.

## 5.5 Computational Cost Comparison

Beyond the training and tuning costs already reported in Section 4.5, the two models differ substantially in size, structural complexity, and inference speed.

Table 9: Computation Cost Comparison - XGBoost vs LSTM

|                                          | <b>XGBoost</b>                                     | <b>LSTM</b>        |
|------------------------------------------|----------------------------------------------------|--------------------|
| Model file size                          | 118.2 KB                                           | 3,352.8 KB         |
| Structural complexity                    | 122 trees,<br>854 total nodes,<br>488 total leaves | 283,905 parameters |
| Inference time (full test set, 528 rows) | 2.61 ms                                            | 172.56 ms          |
| Inference time per row                   | ~4.9 $\mu$ s                                       | 326.8 $\mu$ s      |

**LSTM's** saved model is about **28x larger** than **XGBoost's**, and takes about **66x longer** to produce a single prediction.

LSTM is **much slower** per prediction than XGBoost, especially when considering that XGBoost's inference time was measured on CPU while LSTM's on GPU.

Combined with Section 4.5's figures (XGBoost: 103.1s tuning, 0.06s training; LSTM: 1,966.9s tuning, 90.27s training), **XGBoost** is much **cheaper** on every axis, often by one to three orders of magnitude, while **LSTM** is more **accurate** on every measure in Sections 5.1–5.3.

This isn't a case of one model simply being better. It's a genuine trade-off between accuracy and computational cost.

## 6 Model Compression

### 6.1 Compression Methodology

Following Section 2.5's review, compression is applied to both models, but not identically.

Table 10: Compression Techniques for XGBoost and LSTM

| <b>XGBoost</b>                                                                          | <b>LSTM</b>               |
|-----------------------------------------------------------------------------------------|---------------------------|
| Technique 1: Tree reduction<br>(equivalent of pruning)                                  | Technique 1: Pruning      |
| Technique 2: Precision reduction<br>(equivalent of quantization)                        | Technique 2: Quantization |
| (No Technique 3 — distillation has no natural<br>equivalent for a tree-based ensemble.) | Technique 3: Distillation |
| Combination                                                                             | Combination               |

For each model, a combination of techniques is also tested. Which techniques get combined is decided empirically, based on which individual techniques perform best (Sections 6.2, 6.3).

## 6.2 XGBoost Compression Results

**Technique 1: Tree Reduction** removes boosting rounds from the trained ensemble (122 trees). Testing accuracy at every tree count found a workable trade-off at 86 trees (10% validation MAE tolerance, -29.5% size).

On the test set: **62.5 KB (-47.1%)**, test MAE **28,090 (-5.4% — an improvement)**. This gain isn't a general property of compression — it's the same validation-to-test shift from Section 5.2, which happened to favour this configuration on this test period.

**Technique 2: Precision Reduction** rounds leaf-value outputs to fewer decimal places (split thresholds stay untouched). 2 decimal places was chosen: negligible cost, with a safe margin before accuracy collapses at very low precision.

On the test set: **95.5 KB (-19.2%)**, test MAE **29,746 (+0.2% — negligible)**.

**Combination.** Applying Technique 2 on top of Technique 1 gives a surprising result: the combined file (67.7 KB) is *larger* than Technique 1 alone (62.5 KB), despite the extra compression step. This comes from the fixed serialization overhead noted earlier (Section 6.1) — with only 344 leaves left after tree reduction, the ~10% inflation from reloading/resaving in Python outweighs the small saving from rounding. Accuracy is essentially unchanged (MAE 28,145, -5.2%).

**Summary.** Table 11 compares all three variants against baseline.

Table 11: XGBoost Compression Results per Technique

|                                   | Size     | Δ Size | Test MAE | Δ MAE |
|-----------------------------------|----------|--------|----------|-------|
| Baseline                          | 118.2 KB | —      | 29,691   | —     |
| Technique 1 (Tree reduction)      | 62.5 KB  | -47.1% | 28,090   | -5.4% |
| Technique 2 (Precision reduction) | 95.5 KB  | -19.2% | 29,746   | +0.2% |
| Combination                       | 67.7 KB  | -42.7% | 28,145   | -5.2% |

**Technique 1 wins outright** — smaller *and* more accurate than every other variant, including the Combination — so it's adopted as the final compressed model. Worth noting: XGBoost's baseline was already small (118.2 KB), so this mainly demonstrates the methodology rather than solving a real deployment problem.

Figure 22 visualises this trade-off directly, plotting each variant's size reduction against its accuracy cost.

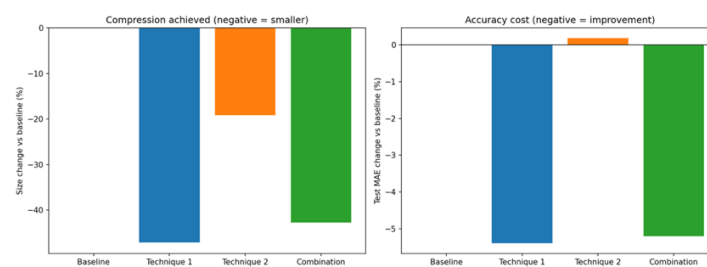


Figure 22: XGBoost - Compression Results and Trade-Off



### 6.3 LSTM Compression Results

Baseline size here is **1,123.3 KB**, not the 3,352.8 KB reported in Section 5.5 — the difference is LSTM's saved Adam optimizer state (momentum/variance buffers), which inflates file size roughly threefold but isn't part of the model's actual learned parameters. It's excluded here for a fair comparison.

**Technique 1: Pruning** sets the least-important half of the model's weights to zero (50% sparsity). This gives **-45.1% size**, but at a real cost: test MAE **+22.0%**. Since pruned weights are zeroed, not removed, computation is unchanged — no inference speedup here, unlike distillation below.

**Technique 2: Quantization** (int8, via TFLite) gives **-71.9% size** for only **+0.8% MAE** — a clearly good trade, with no sharp accuracy cliff like XGBoost's precision sweep (Section 6.2).

**Technique 3: Distillation** trains a smaller "student" network to copy the original 256-unit "teacher." Four student sizes were tested:

Table 12: Candidate Student Sizes test

| Units     | Size $\Delta$ | Test MAE $\Delta$ |
|-----------|---------------|-------------------|
| 32        | -96.4%        | +15.5%            |
| 64        | -91.1%        | +10.1%            |
| <b>96</b> | <b>-83.1%</b> | <b>+7.2%</b>      |
| 128       | -72.2%        | +13.1%            |

Accuracy cost improves steadily from 32 to 96 units, but 128 units is worse than 96 — a genuine overfitting effect, not noise (the same fixed training settings applied to a larger model without extra regularisation). **96 units was selected**: best accuracy of the four, while still over 80% smaller.

**Combination**: quantization applied to the 96-unit distilled student. This gives **-94.4% size** (1,123.3 KB  $\rightarrow$  63.4 KB) at **+6.7% MAE** — beating distillation alone on both counts. This is the opposite of XGBoost's result (Section 6.2), where combining hurt rather than helped.

**Summary.** Table 13 compares all four variants against the clean baseline (Section 6.1).

Table 13: LSTM Compression Results per technique

|                                     | Size       | $\Delta$ Size | Test MAE | $\Delta$ MAE |
|-------------------------------------|------------|---------------|----------|--------------|
| Baseline                            | 1,123.3 KB | —             | 14,854   | —            |
| Technique 1 (Pruning)               | 616.3 KB   | -45.1%        | 18,119   | +22.0%       |
| Technique 2 (Quantization)          | 314.1 KB   | -72.0%        | 14,966   | +0.8%        |
| Technique 3 (Distillation, 96-unit) | 190.2 KB   | -83.1%        | 15,920   | +7.2%        |
| Combination                         | 63.4 KB    | -94.4%        | 15,847   | +6.7%        |

**Pruning is dominated by distillation** (worse on both size and accuracy) and isn't recommended here. **Quantization gives the best accuracy-per-compression trade-off** of any single technique. Unlike XGBoost, there's no single winner: the **Combination** gives the largest compression (-94.4%) at modest cost, while **Quantization alone** gives near-zero accuracy cost (+0.8%) at still-substantial compression (-72.0%). Both are legitimate choices, depending on deployment priorities.

Figure 23 shows the same trade-off for LSTM's four variants, making the Combination's dominant position on the size axis, and Technique 2's dominant position on the accuracy axis, visually apparent.

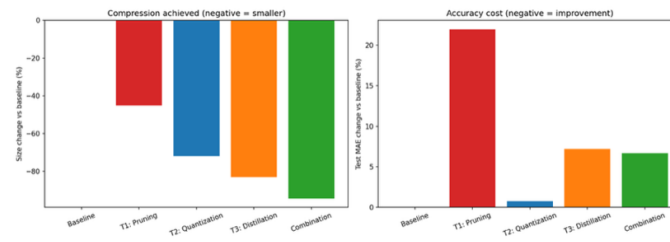


Figure 23: LSTM Compression Results and Trade-Off

## 6.4 Cross-Model Discussion

**Combining techniques helped one model and hurt the other.** XGBoost's Combination was worse than Technique 1 alone; LSTM's was better than any single technique. Same idea, opposite result — combining techniques isn't safely assumed to help; it has to be tested.

**Precision-based techniques were the safest in both models.** XGBoost's precision reduction and LSTM's quantization both gave strong compression at near-zero accuracy cost, matching Khan et al.'s (2026) finding that quantization usually offers the best cost-to-benefit ratio.

**Each model resolved differently.** XGBoost has one clear winner (Technique 1, best on both size and accuracy). LSTM doesn't: the Combination gives the biggest compression, Quantization alone gives the smallest accuracy cost. Both are legitimate, depending on priorities.

**The most interesting result: compression nearly closes the size gap, but not the accuracy gap.** XGBoost's smallest variant is 62.5 KB; LSTM's is 63.4 KB — almost identical, despite LSTM starting ~28x larger. But XGBoost's compressed MAE (28,090) is still far worse than LSTM's (15,847).

This is the clearest evidence for this project's sustainability argument: the accuracy-vs-cost trade-off is real, but **compression** can largely **neutralise the cost** side without giving up the accuracy side.

## 7 Discussion, Conclusion, and Recommendations

### 7.1 Summary of Key Findings

**Predictors.** Weather and calendar effects were strongly confirmed as demand drivers. COVID-19 was confirmed as a genuine, persistent disruption once the seasonal confound was controlled for (Section 3.3.5). Two economic indicators (PMI, RSI) showed no meaningful relationship; this cannot rule out the economic effects on demand and needs future work for investigation.

**Model comparison.** LSTM decisively beat XGBoost on the test set (MAE 14,854 vs. 29,691; MAPE 2.39% vs. 4.88%) (Section 5.2). Both models over-predicted systematically, worst on holidays; a simple correction meaningfully reduced this for both (Section 5.3).

**Explainability.** SHAP confirmed demand-history features dominate both models, matching Section 3.5's correlations — but the two architectures weigh some secondary evidence differently (calendar effects, wind speed), a genuine architectural difference, not two routes to the same answer (Section 5.4).

**Cost and compression.** LSTM's accuracy came at a real cost: ~28x the size and ~66x the inference time of XGBoost (Section 5.5). Compression closed most of this gap: both models' smallest compressed variants ended up almost the same size (62.5 KB vs. 63.4 KB), while

LSTM kept most of its accuracy advantage (Section 6.4) — showing the accuracy-cost trade-off can be largely, though not fully, resolved through compression.

## 7.2 Sustainability Discussion

**Grid decarbonisation and SDG framing.** Section 1.1 positioned this project against SDG 7 and SDG 13: accurate demand forecasting reduces reliance on reserve generation, supporting grid decarbonisation. As **renewables grow (44% of GB generation)**, **accurate forecasting matters more**, not less, given wind and solar's intermittency.

**The accuracy-vs-cost trade-off, resolved rather than restated.** LSTM is more accurate on every metric; XGBoost is cheaper on every computational axis (Section 5.5). Compression narrows this sharply: the size gap nearly closes (62.5 KB vs. 63.4 KB), while LSTM keeps most of its accuracy advantage (Section 6.4). This is the project's clearest sustainability finding — compression can decouple accuracy from cost, rather than forcing a choice, matching the Green AI literature's premise (Khan et al., 2026; Paula et al., 2025).

**A low-cost mitigation worth highlighting.** Section 5.3's bias correction — a constant offset, computed once — improved both models' test accuracy (XGBoost -7.8%, LSTM -2.9%) at effectively zero extra cost. No retraining, no added complexity, none of Section 6's compression cost — arguably the cheapest sustainability win in this project.

**Scope of this evaluation.** This project uses proxy metrics — size, tuning time, training time, inference time — not a full lifecycle carbon assessment. Unlike Paula et al. (2025), who measured actual energy (kWh) and emissions (kg CO<sub>2</sub>e), this project doesn't measure energy draw or data-centre efficiency directly. These are a reasonable proxy for relative efficiency, not a full carbon accounting.

## 7.3 Limitations and Real-World Application

### 7.3.1 Limitations

**Forecasts one day ahead only**, using actual observed weather rather than a weather forecast (Section 4.1). The model **cannot** extend to a **multi-day horizon** without redesigning the pipeline. The accuracy of the forecast also depends on the weather forecast in real-world applications.

**A single fixed train/validation/test split**, not rolling cross-validation. This directly contributes to the validation-to-test shift in Section 5.2 — a real consequence of this design choice, not just an observed curiosity.

**Data ends June 2024.** It missed the recent period and latest trends, for example, the record UK heatwaves of 2025–2026 — the model has never seen a period where an emerging **cooling-driven demand signal** might appear.

**One anomaly remains unexplained.** Both models show their single worst over-prediction on the same day, 12 March 2023 (Section 5.3) — not explained by any available predictor. An acknowledged gap, not a resolved one.

### 7.3.2 Real-World Application

**The daily cycle.** The design is a next-day forecast. In real-world application, by the end of every day, with today's actual demand combined with tomorrow's weather forecast, the model produces tomorrow's forecast. It must **run fresh daily**, not train once and run indefinitely.

**Explainability supports operator trust.** SHAP (Section 5.4) lets operators sanity-check why tomorrow's forecast looks unusually high or low before acting on it — the original motivation for explainability from Section 1.

**Sizes of models may not be the concern.** In absolute terms, even the uncompressed models (118.2 KB and 1,123.3 KB — or 3,352.8 KB before removing the optimizer state) are trivially small for any modern storage or deployment context.

Section 6's compression work is valuable as methodology, but it might not bring practical benefits for this project in real-world application.

## 7.4 Recommendations for Future Work and Conclusion

### 7.4.1 Future Work

**Further economic indicator testing.** PMI and RSI showed no relationship here, but it is still worth further investigating other economic indicators or effects that could contribute as the demand driver.

**Walk-forward or cross-validation splitting,** instead of one fixed validation year — training on an expanding window of past years and validating on each subsequent year in turn, rather than tuning against 2022 alone. This would give more robust hyperparameter selection and test whether Section 5.2's validation-to-test shift is a one-off or a general pattern, at a proportionally higher tuning cost.

**Multi-day forecasting horizon.** Extending beyond one day ahead, for genuine practical use, requires redesigning the lag features and choosing a multi-horizon approach.

**Expanding the data horizon to 2025–2026, to capture the latest trends** — including recent record heatwaves and any emerging cooling-driven demand not present in this dataset.

**Full lifecycle carbon accounting,** following Paula et al.'s (2025) approach — measuring actual energy and emissions directly, rather than the size/time proxies used here (Section 7.2).

### 7.4.2 Conclusion

This project compared XGBoost and LSTM for UK electricity demand forecasting, with LSTM the clearly more accurate model (Section 5.1). Both models were made explainable with SHAP and compressed, showing the accuracy-cost trade-off between them can be largely resolved (Section 6.4).

This supports the project's core premise: sustainability is not just a motivation for this work, but a criterion the models were actually evaluated against.

### References

- Anderson, B. and James, P. (2021) 'Covid-19 lockdown: Impacts on GB electricity demand and CO2 emissions', *Buildings and Cities*, 2(1).
- Bansal, V. (2026) 'Delta-Based Target Reformulation for Short-Term Electricity Load Forecasting Using LSTM and Transformer Models', *arXiv preprint arXiv:2606.17692* [Preprint].
- Bhupatiraju, A. and Ahn, S.B. (2025) 'Explainability-Driven Feature Engineering for Mid-Term Electricity Load Forecasting in ERCOT's SCENT Region', *arXiv preprint arXiv:2507.22220* [Preprint].
- Dong, Q., Huang, R., Cui, C., Towey, D., Zhou, L., Tian, J. and Wang, J. (2025) 'Short-Term Electricity-Load Forecasting by deep learning: A comprehensive survey', *Engineering Applications of Artificial Intelligence*, 154, p. 110980. Available at: <https://doi.org/10.1016/j.engappai.2025.110980>.
- Khan, S., Naz, N.S., Mazhar, T., Tariq, M.U., Shahzad, T., Guizani, S. and Hamam, H. (2026) 'Green AI techniques for reducing energy consumption in AI systems', *Array*, 29, p. 100652. Available at: <https://doi.org/10.1016/j.array.2025.100652>.
- Lee, Y.-G., Oh, J.-Y., Kim, D. and Kim, G. (2023) 'Shap value-based feature importance analysis for short-term load forecasting', *Journal of Electrical Engineering & Technology*, 18(1), pp. 579–588.
- Liu, X. and Lin, Z. (2021) 'Impact of Covid-19 pandemic on electricity demand in the UK based on multivariate time series forecasting with Bidirectional Long Short Term Memory', *Energy*, 227, p. 120455.
- National Energy System Operator (NESO) (2026) 'Britain's Energy Explained: 2025 Review'. National Energy System Operator. Available at: <https://www.neso.energy/news/britains-energy-explained-2025-review> (Accessed: 21 July 2026).
- Paula, E., Soni, J., Upadhyay, H. and Lagos, L. (2025) 'Comparative analysis of model compression techniques for achieving carbon efficient AI', *Scientific Reports*, 15(1), p. 23461.
- Pinheiro, M.G., Madeira, S.C. and Francisco, A.P. (2023) 'Short-term electricity load forecasting—A systematic approach from system level to secondary substations', *Applied Energy*, 332, p. 120493.
- United Nations General Assembly (2015) 'Transforming our world: the 2030 Agenda for Sustainable Development'. United Nations. Available at: <https://sdgs.un.org/2030agenda> (Accessed: 21 July 2026).
- Vivas, E., Allende-Cid, H. and Salas, R. (2020) 'A systematic review of statistical and machine learning methods for electrical power forecasting with reported mape score', *Entropy*, 22(12), p. 1412.

## Appendices

Appendix Table 1: Data Sources and Files

| Category                     | Source                                                                                                                                                                                                     | File Name                                                                        |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Electricity demand           | <a href="https://www.kaggle.com/datasets/albertovidalrod/electricity-consumption-uk-20092022">https://www.kaggle.com/datasets/albertovidalrod/electricity-consumption-uk-20092022</a>                      | historic_demand_2009_2024.csv                                                    |
| Weather                      | <a href="https://www.kaggle.com/datasets/jakewright/2m-daily-weather-history-uk">https://www.kaggle.com/datasets/jakewright/2m-daily-weather-history-uk</a>                                                | all_weather_data.csv                                                             |
| Weather (population weights) | <a href="https://www.nomisweb.co.uk/datasets/pestsyoala">https://www.nomisweb.co.uk/datasets/pestsyoala</a> (Office for National Statistics, "Total population" dataset, via NOMIS — published 26/09/2025) | region_location_lookup.csv<br>(remark: file is the summary from the data source) |
| PMI                          | <a href="https://www.mql5.com/en/economic-calendar/united-kingdom/markit-manufacturing-pmi">https://www.mql5.com/en/economic-calendar/united-kingdom/markit-manufacturing-pmi</a>                          | united-kingdom.markit-manufacturing-pmi.csv                                      |
| RSI                          | <a href="https://www.ons.gov.uk/datasets/retail-sales-index/editions/time-series/versions/45">https://www.ons.gov.uk/datasets/retail-sales-index/editions/time-series/versions/45</a>                      | retail-sales-index-time-series-v45-filtered-2026-06-30T15-15-28Z.csv             |
| COVID stringency             | <a href="https://github.com/OxCGRT/covid-policy-dataset/tree/main">https://github.com/OxCGRT/covid-policy-dataset/tree/main</a>                                                                            | OxCGRT_compact_national_v1.csv                                                   |

Appendix Table 2: UK Regional Populations (2021-06-30)

| Sum of 2021-06-30 (Persons) |                          |                   |
|-----------------------------|--------------------------|-------------------|
| Area code                   | Area name                | Total             |
| E12000008                   | South East               | 9,295,829         |
| E12000007                   | London                   | 8,804,769         |
| E12000002                   | North West               | 7,424,092         |
| E12000006                   | East of England          | 6,349,646         |
| E12000005                   | West Midlands            | 5,956,226         |
| E12000009                   | South West               | 5,712,822         |
| E12000003                   | Yorkshire and The Humber | 5,482,455         |
| S92000003                   | Scotland                 | 5,412,900         |
| E12000004                   | East Midlands            | 4,881,559         |
| W92000004                   | Wales                    | 3,105,633         |
| E12000001                   | North East               | 2,647,493         |
| <b>Grand Total</b>          |                          | <b>65,073,424</b> |

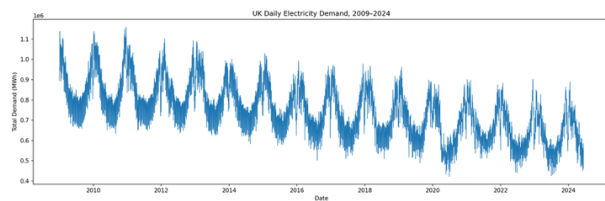
(Source: <https://www.nomisweb.co.uk/datasets/pestsyoala> (Office for National Statistics, "Total population" dataset, via NOMIS — published 26/09/2025))

Appendix Table 3: 11 Selected Weather Stations to Represent Regions

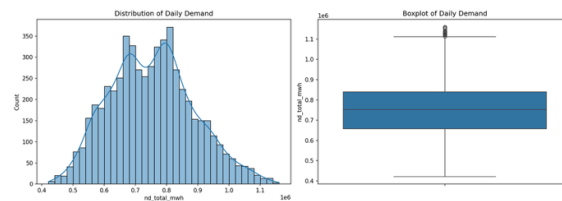
| Region                   | Representative weather station |
|--------------------------|--------------------------------|
| South East               | Crawley                        |
| London                   | London                         |
| North West               | Manchester                     |
| East of England          | Norwich                        |
| West Midlands            | Birmingham                     |
| South West               | Merehead                       |
| Yorkshire and The Humber | Leeds                          |
| Scotland                 | Bishopbriggs                   |
| East Midlands            | Nuneaton*                      |
| Wales                    | Mynachdy                       |
| North East               | Newcastle                      |

(Note: Weather stations were selected based on the nearest most populated city/town. \* East Midlands has no available weather station in the weather dataset and Nuneaton was selected as the closest represented location)

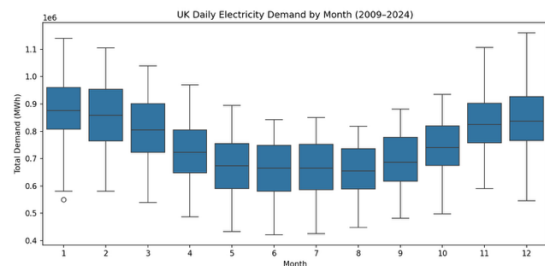
Appendix Figure 1: Daily Demand Trend across 2009-2024



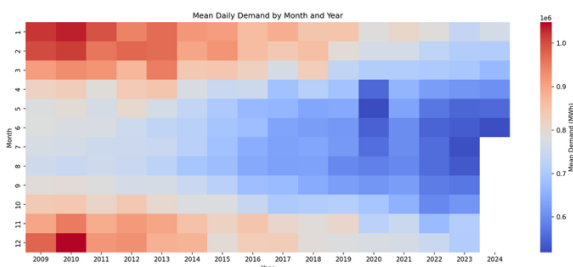
Appendix Figure 2: Distribution of Daily Demand



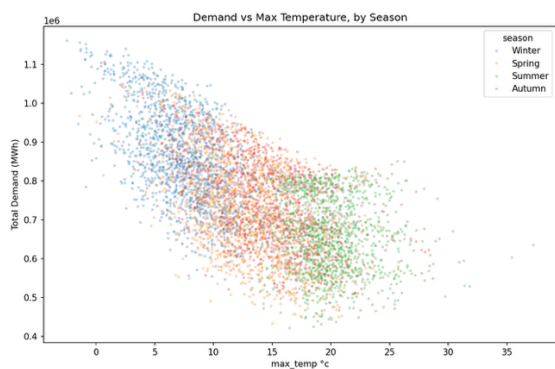
Appendix Figure 3: Daily Demand by Month



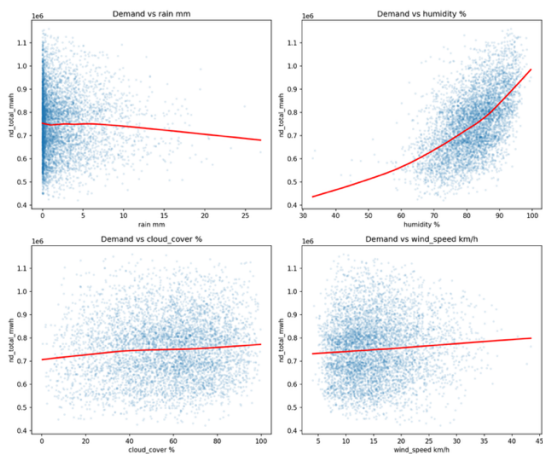
Appendix Figure 4: Mean Daily Demand Heatmap: month-by-year



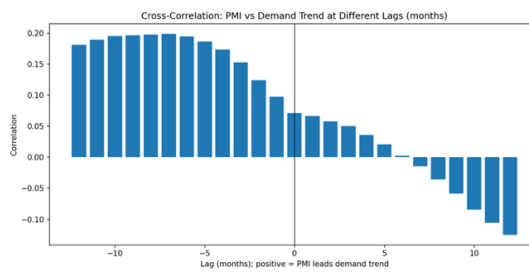
Appendix Figure 5: Demand vs Max Temperature, by Season



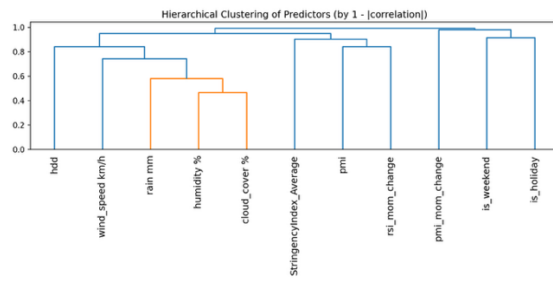
Appendix Figure 6: Demand vs Rain, Humidity, Cloud and Wind Speed



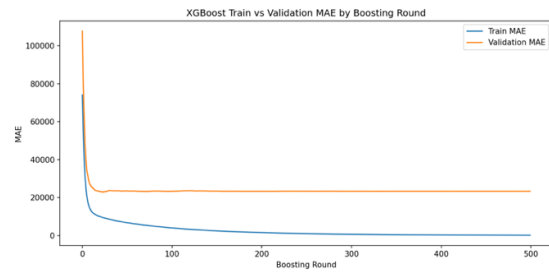
Appendix Figure 7: Cross-Correlation: PMI vs Demand at Different Lags



Appendix Figure 8: Hierarchical Clustering of Predictors



Appendix Figure 9: XGBoost Train vs Validation MAE by Boosting Round



Appendix Table 4: Features and Correlation with Demand

| Feature             | Correlation with demand | Feature                 | Correlation with demand |
|---------------------|-------------------------|-------------------------|-------------------------|
| demand_lag_7        | 0.923                   | is_weekend              | -0.327                  |
| demand_lag_1        | 0.913                   | dow_sin                 | 0.284                   |
| demand_roll_mean_7  | 0.903                   | StringencyIndex_Average | -0.277                  |
| demand_roll_mean_30 | 0.865                   | month_sin               | 0.267                   |
| hdd                 | 0.661                   | is_holiday              | -0.099                  |
| max_temp °c         | -0.645                  | hdd x weekend           | 0.082                   |
| min_temp °c         | -0.624                  | cloud_cover %           | 0.075                   |
| humidity %          | 0.545                   | dow_cos                 | -0.075                  |
| month_cos           | 0.537                   | wind_speed km/h         | 0.059                   |
| demand_roll_std_7   | 0.344                   | rain mm                 | -0.030                  |